VM-GC Integration

Overview

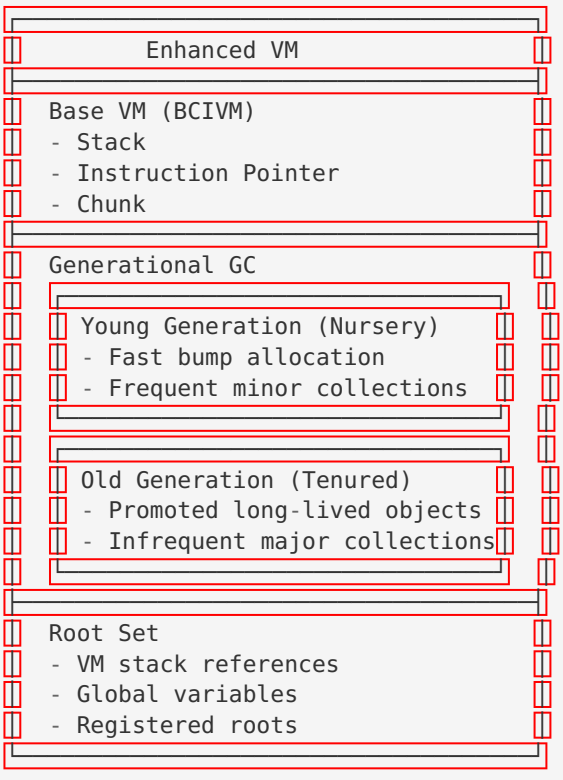
The Enhanced VM now uses the generational garbage collector for all memory allocation, providing automatic memory management for VM objects.

Architecture

Components

- **EnhancedVM**: Extended VM structure with GC integration
- **GenerationalGC**: Two-generation garbage collector (young/old)
- **GCRootSet**: Tracks live object roots from VM stack and globals
- **Write Barriers**: Maintains remembered set for cross-generation references

Memory Layout



API

Initialization

```
// Create VM with default heap split (10% nursery, 90% old)
EnhancedVM* vm = enhanced_vm_create(11 * 1024 * 1024);  // 11MB total

// Create VM with custom sizes
EnhancedVM* vm = enhanced_vm_create_with_sizes(
    1024 * 1024,      // 1MB nursery
    10 * 1024 * 1024  // 10MB old generation
);

// Cleanup
enhanced_vm_destroy(vm);
```

Memory Allocation

```
// Allocate raw memory
void* obj = vm_alloc(vm, size);

// Allocate typed object
void* obj = vm_alloc_object(vm, size, type_id);
```

Allocation Behavior:

- Objects allocated in young generation by default
- Automatic GC triggered if allocation fails
- Returns NULL if out of memory after GC

Garbage Collection

```
// Explicit GC collection
vm_gc_collect(vm);

// Enable/disable GC
enhanced_vm_enable_gc(vm, true);   // Enable
enhanced_vm_enable_gc(vm, false);  // Disable
```

Collection Triggers:

- Automatic: When nursery allocation fails
- Automatic: When old generation reaches 80% capacity
- Explicit: Via `vm_gc_collect()`

Root Management

```
GCObject* obj = (GCObject*)vm_alloc(vm, 100);

// Register as root (prevents collection)
vm_register_root(vm, &obj);

// Use object...

// Unregister when no longer needed
vm_unregister_root(vm, &obj);
```

Write Barriers

```
GenObject* old_obj = (GenObject*)vm_alloc(vm, 100);
GenObject* young_obj = (GenObject*)vm_alloc(vm, 100);

// Promote old_obj to old generation
old_obj->header.generation = GEN_OLD;

// Update reference with write barrier
vm_write_barrier(vm, old_obj, young_obj);
```

Write Barrier Purpose:

- Tracks old→young references
- Maintains remembered set
- Ensures young objects aren’t prematurely collected

Statistics

```
uint64_t collections, allocated, freed;
size_t young_used, old_used;

enhanced_vm_get_gc_stats(
    vm,
    &collections,
    &allocated,
    &freed,
    &young_used,
    &old_used
);

printf("GC Collections: %lu\n", collections);
printf("Bytes Allocated: %lu\n", allocated);
printf("Bytes Freed: %lu\n", freed);
printf("Young Generation: %zu bytes used\n", young_used);
printf("Old Generation: %zu bytes used\n", old_used);
```

Usage Example

```c
#include "vm/vm.h"

int main(void) {
    // Create VM with 1MB nursery, 10MB old generation
    EnhancedVM* vm = enhanced_vm_create_with_sizes(
        1024 * 1024,
        10 * 1024 * 1024
    );

    // Allocate objects
    GCObject* obj1 = (GCObject*)vm_alloc(vm, 100);
    GCObject* obj2 = (GCObject*)vm_alloc(vm, 200);

    // Register important objects as roots
    vm_register_root(vm, &obj1);

    // Allocate many temporary objects
    for (int i = 0; i < 1000; i++) {
        vm_alloc(vm, 50);  // These will be collected
    }

    // Explicit GC (optional, automatic otherwise)
    vm_gc_collect(vm);

    // obj1 survives (registered root)
    // Temporary objects are collected

    // Get statistics
    uint64_t collections;
    enhanced_vm_get_gc_stats(vm, &collections, NULL, NULL, NULL, NULL);
    printf("GC ran %lu times\n", collections);

    // Cleanup
    enhanced_vm_destroy(vm);

    return 0;
}
```

Performance Characteristics

Minor GC (Young Generation)

- **Frequency**: High (triggered on nursery full)
- **Duration**: 1-5ms for 1MB nursery
- **Overhead**: ~10-20ns per allocation
- **Pause**: Stop-the-world

Major GC (Old Generation)

- **Frequency**: Low (triggered at 80% capacity)
- **Duration**: 10-50ms for 10MB old generation
- **Overhead**: Minimal (infrequent)
- **Pause**: Stop-the-world

Allocation Performance

- **Young Generation**: Bump pointer allocation (~10ns)
- **Old Generation**: Free list allocation (~50ns)
- **GC Trigger Overhead**: Amortized across allocations

Memory Overhead

- **Object Header**: 24 bytes per object
- **Root Set**: 8 bytes per root
- **Remembered Set**: 8 bytes per cross-generation reference

Configuration

GC Thresholds

```
// Set GC trigger threshold (percentage of nursery full)
vm->gc_threshold = 80;  // Trigger at 80% (default)

// Set promotion age threshold
generational_gc_set_age_threshold(vm->gc, 5);  // Promote after 5 collections
```

Heap Sizing Guidelines

Small Applications (< 100MB total memory):
- Nursery: 1-2MB
- Old Generation: 10-20MB

Medium Applications (100MB - 1GB):
- Nursery: 10-50MB
- Old Generation: 100-500MB

Large Applications (> 1GB):
- Nursery: 50-200MB
- Old Generation: 1-10GB

Ratio: Typically 10% nursery, 90% old generation

Testing

The integration includes 17 comprehensive tests:

1. **VM-GC Initialization**: Verifies GC setup
2. **Allocation Through GC**: Tests object allocation
3. **Garbage Collection**: Verifies unreachable objects are collected
4. **Write Barriers**: Tests remembered set updates
5. **Minor GC Triggering**: Verifies automatic collection
6. **Root Set Tracking**: Tests root registration/unregistration
7. **Object Promotion**: Verifies young→old promotion
8. **Concurrent Allocation**: Stress test with many allocations
9. **Memory Leak Detection**: Ensures no leaks over time
10. **GC Statistics**: Verifies stat tracking

11. **Explicit GC**: Tests manual collection
12. **Large Object Allocation**: Tests large allocations
13. **Multiple GC Cycles**: Stability over many cycles
14. **Complex Object Graphs**: Tests reachability
15. **GC Disabled Mode**: Tests GC enable/disable
16. **Typed Allocation**: Tests type ID tracking
17. **Default VM Creation**: Tests default heap split

Running Tests

```
cd C
make test_vm_gc_integration
./test_vm_gc_integration
```

Memory Leak Testing

```
valgrind --leak-check=full ./test_vm_gc_integration
```

Sanitizer Testing

```
make clean
CFLAGS="-fsanitize=address -fsanitize=undefined" make test_vm_gc_integration
./test_vm_gc_integration
```

Implementation Notes

Current Limitations

1. **Root Scanning**: Currently placeholder - will be extended when VM has proper object types
2. **Object References**: VM stack contains doubles, not object pointers yet
3. **Compaction**: Not yet implemented for old generation
4. **Concurrent GC**: Not yet implemented (all collections are stop-the-world)

Future Enhancements

1. **Incremental GC**: Reduce pause times with incremental collection
2. **Concurrent Marking**: Mark phase runs concurrently with mutator
3. **Generational Barriers**: Optimize write barriers for common patterns
4. **Adaptive Sizing**: Automatically adjust heap sizes based on usage
5. **Object Pinning**: Pin objects for FFI/native code interaction

Troubleshooting

Out of Memory Errors

```
VM: Out of memory (requested X bytes)
```

Solutions:
- Increase heap size in `enhanced_vm_create_with_sizes()`

- Reduce object allocation rate
- Ensure objects are not unnecessarily kept alive

Excessive GC Pauses

Symptoms: Frequent GC collections, high pause times

Solutions:
- Increase nursery size to reduce minor GC frequency
- Adjust promotion threshold to keep objects in nursery longer
- Profile allocation patterns to identify hotspots

Memory Leaks

Symptoms: Memory usage grows over time

Solutions:
- Ensure all roots are properly unregistered
- Check for circular references in object graphs
- Use valgrind to identify leak sources

References

- **Generational GC Paper**: “Garbage Collection in an Uncooperative Environment” (Boehm, 1988)
- **Write Barriers**: “A Real-Time Garbage Collector with Low Overhead” (Baker, 1978)
- **VM Design**: “Crafting Interpreters” (Nystrom, 2021)

Related Documentation

- `C/vm/gc/README.md` - Generational GC implementation details
- `docs/phase8/PHASE8_PLAN.md` - Phase 8 integration plan
- `docs/phase8/INTEGRATION_POINTS.md` - System integration specifications